

Gestione dei file

Il file è l'unità logica con la quale è possibile la memorizzazione **persistente** di dati su memoria di massa.

Nel linguaggio C è possibile creare, aprire, leggere, scrivere (in generale manipolare) i file utilizzando le funzioni standard definite e contenute nella libreria I/O (<stdio.h>).

Ogni programma vede un file come una sequenza di **record logici** (singolo carattere, parola, oppure intera riga), che termina con un indicatore di fine file (EOF – End Of File). I file sono gestiti dal Sistema Operativo, pertanto le funzioni di I/O definite nella libreria sopra indicata terranno conto del S.O. ospitante (della macchina).

I file in C vengono suddivisi in due categorie:

1. **file di testo**, trattati come sequenze di caratteri, organizzati in righe, ciascuna delle quali termina con un '\n';
2. **file binari**, trattati come sequenze di bit.

I file hanno una struttura sequenziale; questo significa che per poter accedere ad un particolare record logico, è necessario prima scorrere tutti quelli che lo precedono (i record sono organizzati in una sequenza rigidamente ordinata).

In C per manipolare un file è necessario andare a dichiarare una variabile che lo rappresenti (**puntatore a file**); questa indica implicitamente il file al quale stiamo facendo riferimento e anche l'elemento corrente all'interno della sequenza. La variabile che rappresenta il file sarà di tipo FILE, che è un tipo di dato non primitivo, dichiarato nella libreria I/O.

Prima di poter manipolare un file, è necessario aprirlo. Per farlo utilizziamo la funzione fopen().

Sintassi:

```
puntatore = fopen(nomeFile, modalità);
```

nomeFile rappresenta il nome del file che si vuole creare/aprire, mentre *modalità* è un singolo carattere che indica in che modo intendiamo aprire il file in questione. Le possibilità per quest'ultimo parametro sono le seguenti:

- "w": il file viene aperto in modalità scrittura (se il file non esiste, questo viene creato);
- "a": il file viene aperto in modalità **append**, ovvero scrittura in coda al file (se il file non esiste, questo viene creato);

- "r": il file viene aperto in modalità lettura.
- "b": a fianco di una delle precedenti, indica che il file è binario (di default, se si omette, il file è considerato come file testuale).

L'operazione di apertura del file inizializza il puntatore a tale file: se avviene con successo, restituisce un puntatore al file aperto, che verrà usato come riferimento per accedere e manipolare tale file.

Esempio:

```
FILE *mioFile;

mioFile = fopen("nomeFile.txt", "w");
```

N.B.: se non diversamente specificato, la funzione fopen() apre/crea il file nella stessa cartella del file sorgente. Se vogliamo scegliere un'altra cartella, è opportuno inserire il percorso assoluto fino a quella cartella.

Una volta che abbiamo terminato di utilizzare il file, è necessario chiuderlo utilizzando la funzione fclose(). È molto importante farlo per far sì che tutti i salvataggi avvengano correttamente, che altri programmi possano utilizzare il file e per ripulire memoria che non viene più utilizzata.

La funzione fclose() restituisce come risultato un intero:

- se l'operazione di chiusura è avvenuta correttamente restituisce 0;
- se la chiusura non è andata a buon fine, restituisce la costante EOF (tipicamente -1).

Esempio:

```
FILE *mioFile;

mioFile = fopen("nomeFile.txt", "w");

fclose(mioFile);
```

Nota

La funzione fopen() restituisce NULL se l'operazione di apertura del file fallisce. I motivi principali per i quali l'apertura di un file non avviene con successo sono l'assenza del file (il file che si cerca di aprire non esiste nella cartella) oppure il tentativo di aprire il file in una modalità non consentita (ad esempio tentare di aprire in scrittura un file in sola lettura).

Pertanto, è possibile controllare (tramite if) se l'apertura del file ha avuto successo oppure no e tentare di leggere solamente se si è certi che il file è stato aperto correttamente.

Esempio:

```
FILE *mioFile;

mioFile = fopen("nomeFile.txt", "r");

if(mioFile != NULL){
    //operazioni sul file
}
else{
    printf("Impossibile trovare il file!");
}

fclose(mioFile);
```

Esistono tre file testuali, che sono standard per l'I/O, e che sono aperti automaticamente ogni volta che avviamo un programma:

1. **stdin**: lo standard input (ad esempio la tastiera), aperto in lettura;
2. **stdout**: lo standard output (ad esempio il monitor), aperto in scrittura;
3. **stderr**: lo standard error (ad esempio il monitor), aperto in scrittura.

Queste tre variabili sono dei puntatori a file che vengono definiti in maniera del tutto automatica ed implicita, pertanto non vanno dichiarati.

Scrittura di un file

Come abbiamo detto poco sopra, è possibile aprire un file in scrittura indicando il valore "w" come secondo parametro della funzione fopen().

Per poter scrivere su un file, dopo averlo aperto, è possibile utilizzare la funzione fprintf().

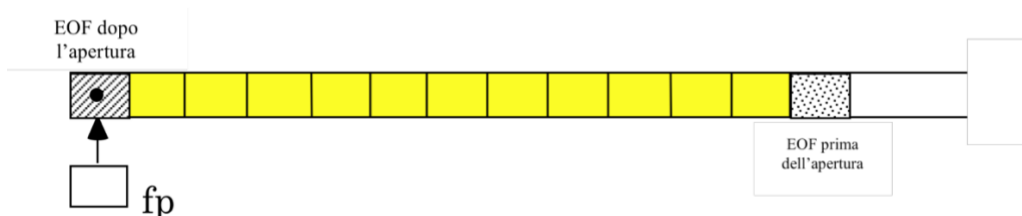
Sintassi:

```
fprintf(puntatore, stringa_controllo, elem);
```

dove *puntatore* è il puntatore al file sul quale vogliamo scrivere (che deve già essere stato aperto in scrittura), *stringa_controllo* specifica il formato

dei dati da scrivere, ed *elem* è l'elenco dei valori che vogliamo scrivere sul file.

Aperto un file in questa modalità, posizioneremo il puntatore al file all'inizio del file stesso (sia che sia vuoto oppure no): questo significa che se il file conteneva già del testo, il nuovo testo andrà a sostituire quello vecchio (sovrascrittura).



Esempio:

```
FILE *mioFile;  
int c = 0;  
  
mioFile = fopen("nomeFile.txt", "w");  
  
fprintf(mioFile, "Risultato: %d", c + 1);  
  
fclose(mioFile);
```

Questa funzione restituisce il numero di elementi scritti su file, oppure un valore negativo in caso di errore.

Possiamo chiederci come evitare di eliminare il contenuto del nostro file.

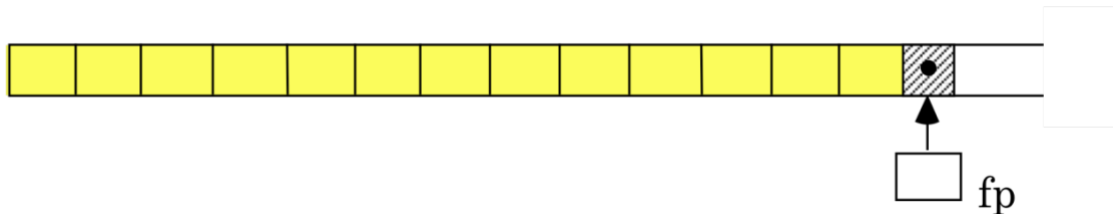
Per questo è sufficiente aprire il file in modalità append ("a" come secondo parametro di `fopen()`); in questo modo andremo a posizionare il puntatore al file in fondo al file stesso (dopo l'ultimo record significativo) e, quindi, scriveremo in fondo al file, senza sovrascrivere il contenuto preesistente.

Esempio:

```
FILE *mioFile;  
  
mioFile = fopen("nomeFile.txt", "a");
```

```
fprintf(mioFile, "\nBella giornata di sole!");  
  
fclose(mioFile);
```

In questo esempio, se riapriamo lo stesso file di prima in modalità append, il nuovo testo verrà aggiunto in coda a quello precedente, senza sovrascriverlo.



Letture di un file

La lettura di un file può avvenire aprendolo attraverso la funzione `fopen()`, specificando "r" come secondo parametro.

La lettura di un file è leggermente più complicata rispetto alla scrittura.

Per prima cosa è necessario creare una variabile di tipo stringa (un array di caratteri), con la quale memorizzeremo il contenuto del file.

Possiamo utilizzare la funzione `fgets()`. Abbiamo già incontrato questa funzione, quando abbiamo visto che può anche essere utilizzata per leggere una stringa da tastiera. Ricordiamo che questa funzione ha bisogno di tre parametri:

- la variabile che andrà a contenere il testo che andremo a leggere (dal file);
- la grandezza massima dei dati che andremo a leggere (deve essere pari alla dimensione della stringa)
- il puntatore dal quale leggere.

Esempio:

```
FILE *mioFile;  
  
mioFile = fopen("nomeFile.txt", "r");  
  
char riga[100];
```

```
fgets(riga, 100, mioFile);

printf("%s", riga);

fclose(mioFile);
```

La funzione fgets() ci consente di leggere **una sola riga** del nostro file.

Quindi se il file contiene più righe di testo, è opportuno utilizzare un ciclo while:

```
FILE *mioFile;

mioFile = fopen("nomeFile.txt", "r");

char riga[100];

while(fgets(riga, 100, mioFile)){
    printf("%s", riga);
}

fclose(mioFile);
```

Il ciclo while continuerà ad essere eseguito fintanto che vengono trovate delle righe all'interno del file.

Un altro modo possibile è utilizzare la funzione fscanf(), che serve per leggere dati formattati da file (in maniera simile a scanf() che leggeva da tastiera).

Sintassi:

```
fscanf(puntatore, stringa_controllo, ind_elem);
```

dove *puntatore* è il puntatore al file aperto e *stringa_controllo* serve ad indicare il formato dei dati che vogliamo leggere (%d, %f, ...), ed *ind_elem* è un elenco degli indirizzi delle variabili dove memorizzare i dati (esattamente come nella classica scanf()).

Questa funzione restituisce il numero di elementi letti da file, se ha successo, oppure la costante EOF in caso di fallimento o di fine file.

Esempio:

```
FILE *fp = fopen("dati.txt", "r");
int x; float y;
fscanf(fp, "%d %f", &x, &y);
fclose(fp);
```

In questo esempio vengono letti due valori dal file *dati.txt*, uno di tipo intero e uno di tipo float.

Possiamo osservare che:

`printf(stringa_controllo, elementi);`

`scanf(stringa_controllo, elementi);`

equivalgono a:

`fprintf(stdout, stringa_controllo, elementi);`

`scanf(stdin, stringa_controllo, elementi);`